

// **React Theory Questions**

// *What is React? Why use it?

// *What are components in React?

// *What is JSX?

// *Explain virtual DOM and how it works.

// *What are props and state?

// *What is the difference between functional and class components?

// *What is the useEffect Hook? How is it different from componentDidMount?

// *What are controlled vs uncontrolled components?

// *What is prop drilling? How to avoid it?

// *What are keys in React? Why are they important?

// *What is React.memo and how does it help performance?

// *What is the reconciliation process in React?

// *How does React handle re-rendering?

// *What is context API and when should it be used?

// * Explain the React lifecycle methods.

// *What are HOCs (Higher-Order Components)?

// *What is React Fiber?

// *What are render props?

1. **What is the Virtual DOM and how does React use it for rendering?**

What is the Virtual DOM?

- The DOM (Document Object Model) is the browser's tree-like structure that represents all elements of a webpage.
- Directly updating the real DOM is slow, because each change forces the browser to re-render parts of the UI (layouts, styles, painting, etc.).
- To solve this, React uses a Virtual DOM (VDOM):
 - o It's a lightweight JavaScript object that is a copy of the real DOM.
 - o Think of it as a "blueprint" of the UI stored in memory.

◆ How React uses the Virtual DOM for Rendering

1. Initial Render

- o When you first render a React component, React creates a Virtual DOM tree.
 - o It then renders that structure to the real DOM only once.
 - 2. State/Props Update
 - o When data changes (state or props), React rebuilds a new Virtual DOM for that component tree.
 - 3. Diffing Algorithm (Reconciliation)
 - o React compares the new Virtual DOM with the previous Virtual DOM (using an efficient algorithm called diffing).
 - o It identifies the minimal set of changes needed.
 - 4. Update the Real DOM
 - o React applies only those changes to the actual DOM (not the entire UI).
 - o This makes updates faster and more efficient.
-

◆ Benefits

- Performance: Avoids unnecessary re-rendering of the real DOM.
- Declarative UI: Developers describe what the UI should look like, and React handles the efficient updates.
- Predictability: Makes UI updates more consistent.

2. How does React diff changes and batch updates efficiently?

React's Diffing Algorithm (Reconciliation)

When state or props change:

1. React builds a new Virtual DOM tree.
2. It compares (diffs) the new Virtual DOM with the previous version.
3. Instead of replacing the whole DOM, it calculates the minimum changes needed.

Diffing Rules React Uses:

- Element type check:
 - o If the element type is the same (`<div>` → `<div>`), React reuses the existing DOM node and updates attributes.
 - o If the element type is different (`<div>` → `<span>`), React destroys the old node and creates a new one.
- Component diffing:
 - o If the component type is the same, React re-renders it with updated props/state.
 - o If the type changes, React discards the old component and mounts a new one.

- List diffing (Keys):
 - o React uses key props to efficiently match elements in arrays.
 - o With keys, React reuses nodes instead of re-rendering the whole list.
 - o Example:
 - o `items.map(item => <li key={item.id}>{item.text}</li>)`

Without keys, React may destroy/recreate list items unnecessarily.

Batching Updates

Normally, if you run multiple `setState` (or state updates with Hooks), React batches them into one update for performance.

Example:

```
setCount(count + 1);
```

```
setName("Krishna");
```

- Instead of updating the DOM twice, React batches them and updates once after the event handler finishes.

How React Batching Works:

- Event Handlers (default batching): React batches state updates automatically inside event handlers.
- Async code (like promises, `setTimeout`): Before React 18, updates were not batched here. Since React 18, automatic batching works everywhere (event handlers, promises, `async/await`, timeouts).

Why This is Efficient

- Diffing ensures minimal DOM operations (DOM changes are costly).
- Batching ensures fewer re-renders (multiple state updates → one DOM update).
- Combined, React avoids unnecessary work and keeps apps fast.

✅ In short:

React first diffs the new Virtual DOM against the old one to find minimal changes, then batches multiple updates together to apply them efficiently in one go.

What happens during the React rendering and reconciliation phase?

1. Render Phase (a.k.a. Reconciliation Phase)

This is the "thinking" phase 🧠. React figures out what needs to change.

- Steps:

1. The component function (or `render()` in class components) is called.

2. React builds a new Virtual DOM tree (React elements / Fiber tree).
 3. React compares the new Virtual DOM with the previous Virtual DOM (using the diffing algorithm).
 4. React produces a list of changes (effects) that need to be applied to the real DOM.
 - Key points:
 - o Purely a calculation phase → no actual DOM changes yet.
 - o Can be paused, aborted, or restarted (with React Fiber).
 - o This is why React is considered asynchronous — it doesn't block the UI while diffing.
-

2. Commit Phase

This is the "doing" phase ⚡. React applies the changes to the real DOM.

- Steps:
 1. React takes the list of changes (diff result).
 2. It updates the real DOM nodes.
 3. Runs lifecycle methods/effects:
 - Class components: `componentDidMount`, `componentDidUpdate`.
 - Function components: `useLayoutEffect`, `useEffect`.
 - Key points:
 - o This phase is synchronous (it cannot be interrupted).
 - o DOM updates and screen painting happen here.
-

◆ React Fiber's Role

React Fiber (introduced in React 16) is the underlying engine that:

- Splits the render/reconciliation phase into small units of work.
 - Lets React pause and resume rendering for better responsiveness.
 - Supports concurrent rendering (React 18+).
-

◆ Summary (Easy Flow)

1. Render Phase:
 - o Build new VDOM → Diff with old VDOM → Prepare changes.
 - o (Async, can be paused).
2. Commit Phase:
 - o Apply changes to real DOM → Run lifecycle hooks/effects.

- o (Sync, can't be paused).

What's the role of the Fiber architecture?

Why Fiber Was Introduced

Before React 16, React's reconciliation worked in a synchronous stack-based way:

- When React started rendering, it would render the whole component tree recursively.
- For large trees, this could block the main thread → the UI would "freeze" until React finished.

👉 To fix this, React introduced Fiber in React 16.

♦ What is Fiber?

- A Fiber is a JavaScript object that represents a unit of work for a component.
- Think of it as a data structure that keeps track of:
 - o The component's type and props.
 - o Its state and hooks.
 - o Its place in the tree (parent/child/sibling).
 - o Side effects (what needs to happen in commit phase).

In short: Fiber is the new reconciliation engine that breaks rendering into small pieces.

♦ Role of Fiber Architecture

1. Interruptible Rendering

- o Fiber allows React to pause work on one part of the tree and switch to more urgent tasks (like responding to user input).
- o This makes React asynchronous and non-blocking.

2. Prioritization of Updates

📌 Fiber assigns priorities to updates:

📌 High priority: User interactions (typing, clicks).

Low priority: Background data loading, off-screen rendering.

- o React can work on urgent updates first, then come back to less urgent ones.

3. Incremental Rendering

- o Instead of rendering the whole tree at once, Fiber breaks it into units of work.
- o Each Fiber node can be processed independently.
- o If React needs to stop (say, for a user scroll), it can resume later.

4. Better Scheduling

- o React uses a cooperative scheduler: it yields control back to the browser between units of work.
 - o This keeps the app responsive.
5. Foundation for Concurrent Mode (React 18+)
- o Features like `useTransition`, `Suspense`, and concurrent rendering are possible only because of Fiber.

◆ Fiber vs Old Reconciliation

Feature	Old Stack Reconciler	Fiber Reconciler
Rendering	Synchronous, recursive	Asynchronous, incremental
Pausable?	✗ No	✓ Yes
Prioritization	✗ No	✓ Yes
Concurrent features	✗ Not supported	✓ Supported
Data structure	Function stack	Fiber tree (linked list)

◆ Simple Analogy

Imagine React rendering as doing chores:

- Old React: You must finish cleaning the entire house before answering the door. 🏠
- React Fiber: You can clean room by room, pause if the doorbell rings, answer it, then continue later.

How do React hooks like `useEffect` and `useLayoutEffect` differ?

1. `useEffect`

- When it runs:

After React has painted the UI to the screen (asynchronous, after the commit phase).

- Usage:

For non-blocking side effects that don't affect the visible layout.

- Examples:

- o Fetching data (`fetch`, `axios`).
- o Setting up subscriptions (`WebSocket`, event listeners).
- o Logging, analytics.

- Why:

Since it runs after painting, it doesn't block the browser from updating the UI → keeps the app responsive.

```
useEffect(() => {
```

```
console.log("Runs after paint");

document.title = `Count: ${count}`;

}, [count]);
```

◆ 2. useEffect

- When it runs:

Immediately after React updates the DOM but before the browser paints the screen (synchronous, still in commit phase).

- Usage:

For synchronous side effects that must run before the user sees the update.

- Examples:

- o Measuring DOM nodes (getBoundingClientRect).
- o Synchronously mutating styles or scroll position.
- o Animations that need precise initial measurements.

- Why:

Since it runs before paint, the user won't see flickers (layout shifts).

```
useEffect(() => {

  console.log("Runs before paint");

  const rect = divRef.current.getBoundingClientRect();

  console.log("Element size:", rect);

}, []);
```

◆ Key Differences

Feature	useEffect 🕒	useLayoutEffect ⚡
Timing	After paint (async)	Before paint (sync)
Blocks visual updates?	❌ No	✅ Yes
Use case	Non-UI side effects (data, logs, subscriptions)	DOM measurements, style/layout adjustments
Performance	More performant, doesn't block paint	Can cause jank if overused

◆ Rule of Thumb

- ✅ Use useEffect by default.

-  Use `useLayoutEffect` only when you need to measure or modify DOM layout before the user sees it.
-  Don't overuse `useLayoutEffect` → it can block painting and hurt performance.

What are controlled vs uncontrolled components in forms?

// → When would you use each, and why?

◆ Controlled Components

- A controlled component is a form element (like `<input>`, `<textarea>`, `<select>`) whose value is controlled by React state.
- The React component is the “single source of truth”.

Example:

```
function ControlledInput() {
  const [name, setName] = React.useState("");

  return (
    <input
      type="text"
      value={name}
      onChange={(e) => setName(e.target.value)}
    />
  );
}
```

- Here:
 - o `value={name}` → React decides what's shown.
 - o Every keystroke triggers `onChange`, which updates state → rerenders → updates input.

Pros:

- Easy to validate, transform, or restrict input in real-time.
- Keeps form data centralized in React state → predictable.
- Works well for complex forms (with validation, conditional logic).

Cons:

- More boilerplate code (handlers, state).
 - Slightly more overhead for large forms.
-

◆ Uncontrolled Components

- An uncontrolled component is a form element that manages its own state internally in the DOM.
- React doesn't control the value directly. Instead, you access it using a ref when needed.

Example:

```
function UncontrolledInput() {  
  const inputRef = React.useRef();  
  
  function handleSubmit() {  
    alert(inputRef.current.value); // Read directly from DOM  
  }  
  
  return (  
    <>  
      <input type="text" ref={inputRef} />  
      <button onClick={handleSubmit}>Submit</button>  
    </>  
  );  
}
```

- Here:
 - o Input value is stored in the DOM.
 - o React only grabs it when needed via ref.

✅ Pros:

- Less code, simpler setup for small forms.
- Useful for quick uncontrolled data entry (like search boxes).
- Can integrate with non-React code (jQuery, legacy scripts).

❌ Cons:

- Harder to enforce validation or formatting while typing.
- Multiple sources of truth (DOM + React state).
- Not ideal for complex, dynamic forms.

◆ When to Use Each?

Use Controlled Components when:

- You need real-time validation (e.g., password strength meter).

- You want to conditionally enable/disable UI based on input.
- You need to save form data in React state for later use (e.g., multi-step forms, wizard-style forms).

Use Uncontrolled Components when:

- You just need the value once on submit, not on every keystroke.
- You're working with simple forms or migrating from legacy DOM code.
- You want less boilerplate and don't care about React tracking every keystroke.

What causes unnecessary re-renders in React and how can you optimize them?

// → How can React.memo, useMemo, and useCallback help?

What Causes Unnecessary Re-renders in React?

React re-renders a component when:

1. Parent component re-renders (children re-render unless optimized).
2. Props change (even if the new value is referentially different but logically the same, like a new object/array/function).
3. `<Child data={{ name: "Krishna" }} />` // new object every render
4. State updates (even if setting the same value again).
5. Context updates (all consumers re-render on context value change).
6. Anonymous functions / inline objects inside JSX (new reference each render).

◆ How to Optimize Re-renders?

1. React.memo

- Wraps a functional component.
- Prevents re-rendering if props haven't changed (shallow comparison).

```
const Child = React.memo(({ name }) => {
  console.log("Child rendered");
  return <div>{name}</div>;
});
```

```
function Parent() {
  const [count, setCount] = React.useState(0);
  return (
    <>
      <button onClick={() => setCount(count + 1)}>+</button>
    </>
  );
}
```

```

    <Child name="Krishna" /> { /* Won't re-render unnecessarily */ }
  </>
);
}

```

✅ Best for pure components where props rarely change.

⚠️ Doesn't help if you pass new objects/functions every render.

2. useMemo

- Memoizes the result of a computation.
- Prevents recalculating expensive values on every render.

```

const filteredList = useMemo(() => {
  return items.filter(item => item.active);
}, [items]);

```

✅ Best for expensive calculations or derived data.

⚠️ Don't overuse — memoization itself has overhead.

3. useCallback

- Memoizes a function definition.
- Prevents creating a new function reference on every render.

```

const handleClick = useCallback(() => {
  console.log("Clicked");
}, []); // function reference stays stable

```

- Without useCallback, passing functions to children causes re-renders (because a new function reference is created every time).
- With useCallback, child components wrapped in React.memo can skip re-renders.

✅ Best when passing callbacks to memoized children.

◆ Quick Comparison

Hook / Tool	Prevents	Best Use Case
React.memo	Child re-renders due to unchanged props	Pure components
useMemo	Re-computing expensive values	Derived data, filtering, sorting
useCallback	Re-creating functions on each render	Passing stable callbacks to children

◆ Other Optimization Techniques

- Key usage in lists → Use stable key to avoid unnecessary re-mounts.
- Split large components → Smaller components re-render less often.
- Context optimization → Split contexts or use libraries like zustand or jotai for fine-grained updates.
- Windowing/Lazy rendering → For big lists, use libraries like react-window.

How does context API compare to Redux for state management?

// → When is one preferable over the other?

1. React Context API

- What it is:

A built-in React feature for prop drilling avoidance. Lets you share data globally (theme, user, language, auth, etc.) without manually passing props down the tree.

- How it works:

- o Create a context → Wrap part of your app with a Provider → Any component inside can consume the value with useContext.

```
const ThemeContext = React.createContext("light");
```

```
function App() {  
  return (  
    <ThemeContext.Provider value="dark">  
      <Child />  
    </ThemeContext.Provider>  
  );  
}
```

```
function Child() {  
  const theme = React.useContext(ThemeContext);  
  return <p>Theme: {theme}</p>;  
}
```

- Pros:
 - o Simple, built-in (no extra library).
 - o Great for small/medium apps.
 - o Perfect for static or low-frequency data (theme, locale, auth).

- Cons:
 - o Every context update re-renders all consumers (can cause performance issues in large trees).
 - o No built-in devtools for debugging.
 - o No opinionated structure (easy to misuse for complex state).
-

◆ 2. Redux

- What it is:

A state management library for predictable global state.

Uses a single store, actions, and reducers to manage state updates.

- How it works:

o You dispatch an action → Reducer updates the state → Components subscribed to that slice re-render.

// slice with Redux Toolkit

```
const counterSlice = createSlice({
  name: "counter",
  initialState: 0,
  reducers: {
    increment: state => state + 1,
    decrement: state => state - 1
  }
});
```

```
const { increment } = counterSlice.actions;
```

- Pros:
 - o Scales well for large/complex apps.
 - o Predictable: pure reducers, strict flow.
 - o Excellent devtools (time travel debugging, logging).
 - o Ecosystem support: middleware (redux-thunk, redux-saga), persistence, etc.
 - o Fine-grained re-render control (connect, selectors).
- Cons:
 - o More boilerplate than Context (though Redux Toolkit reduced this a lot).
 - o Learning curve for beginners.
 - o Can be "overkill" for small apps.

◆ When to Use Which?

✅ Prefer Context API when:

- Your app is small/medium.
- You just need to avoid prop drilling.
- Global state updates are infrequent (theme, auth, language).
- You want zero extra dependencies.

✅ Prefer Redux when:

- Your app is large, complex, or collaborative.
- You need predictable state flow with debugging tools.
- You have frequent or complex state updates (e.g., forms, API data, caching, undo/redo).
- You want middleware support (logging, async actions, side effects).
- You need to optimize performance (Redux selectors prevent unnecessary re-renders).

◆ Quick Comparison

Feature	Context API 	Redux 
Built-in?	✅ Yes ❌ No (external lib)	
Setup Complexity	Low	Medium/High
Debugging Tools	❌ Limited	✅ Powerful DevTools
Best for	Small apps, simple global state	Large apps, complex state logic
Performance on updates	Re-renders all consumers	Fine-grained control via selectors
Async logic handling	Manual (useEffect)	Middleware (redux-thunk, saga, etc.)

What is prop drilling and how do you avoid it?

// → What design patterns help in large applications?

What is Prop Drilling?

- Prop drilling happens when you pass props through multiple layers of components just so a deeply nested child can use them.
- The intermediate components don't care about the data but still have to pass it down.

Example:

```
function App() {  
  const user = { name: "Krishna" };  
}
```

```
    return <Parent user={user} />;  
  }  
}
```

```
function Parent({ user }) {  
  return <Child user={user} />;  
}
```

```
function Child({ user }) {  
  return <GrandChild user={user} />;  
}
```

```
function GrandChild({ user }) {  
  return <p>Hello {user.name}</p>;  
}
```

👉 Here, user prop is drilled through Parent → Child → GrandChild even though only GrandChild uses it.

◆ Why is Prop Drilling a Problem?

- **Boilerplate:** Passing props through many levels.
- **Coupling:** Intermediate components depend on props they don't use.
- **Maintainability:** If data changes, many components need updates.
- **Scalability issue:** Large apps with deeply nested trees become hard to manage.

◆ How to Avoid Prop Drilling?

1. React Context API (Built-in)

- Provides a way to pass data directly to any component in the tree without prop drilling.

```
const UserContext = React.createContext();
```

```
function App() {  
  const user = { name: "Krishna" };  
  return (  
    <UserContext.Provider value={user}>  
      <GrandChild />  
    </UserContext.Provider>  
  )  
}
```

```
);  
}
```

```
function GrandChild() {  
  const user = React.useContext(UserContext);  
  return <p>Hello {user.name}</p>;  
}
```

✅ Great for themes, auth, language.

⚠️ Not ideal for very frequent updates (causes re-renders).

2. State Management Libraries (Redux, Zustand, Jotai, Recoil)

- Provide a global store so components can read/write state without drilling props.
 - Useful when you have complex app state (caching, multiple data sources, async updates).
-

3. Component Composition

- Instead of passing props down many layers, pass children as functions/components.

```
function Layout({ header, content }) {  
  return (  
    <div>  
      <header>{header}</header>  
      <main>{content}</main>  
    </div>  
  );  
}
```

```
function App() {  
  return (  
    <Layout  
      header={<Header />}  
      content={<Dashboard />}  
    />  
  );  
}
```


- ✓ Reduces unnecessary props.
-

4. Custom Hooks

- Extract shared logic into hooks so you don't need to pass props everywhere.

```
function useUser() {  
  return React.useContext(UserContext);  
}
```

```
function Profile() {  
  const user = useUser();  
  return <p>{user.name}</p>;  
}
```

5. Event Emitters / Pub-Sub Pattern

- Useful for loosely coupled communication between components.
 - Instead of drilling, a child can emit an event, and another component can listen.
 - Often replaced by context or state management libs in React.
-

◆ Design Patterns for Large Applications

- Context for static/global values (auth, theme, locale).
- Redux/Zustand/Recoil for complex state with frequent updates.
- Compound Components Pattern (pass only what's needed via composition).
- Custom Hooks for reusing business logic.
- Colocation (keep state as close as possible to where it's needed to avoid lifting state unnecessarily).

How do keys work in lists and why are they important?

// → What happens if keys are not unique?

◆ What are Keys in Lists?

- A key is a special prop React uses to identify which items in a list have changed, added, or removed.
- It helps React's reconciliation algorithm (diffing) decide how to update the DOM efficiently.

Example:

```
const items = ["A", "B", "C"];
```

```
function MyList() {  
  return (  
    <ul>  
      {items.map((item, index) => (  
        <li key={item}>{item}</li>  
      ))}  
    </ul>  
  );  
}
```

Here, each `<li>` has a key (A, B, C).

◆ Why are Keys Important?

Without keys, React has to re-render everything blindly because it can't track which item is which.

With keys:

- React reuses DOM nodes for unchanged items.
 - Only creates/removes/updates the nodes that changed.
 - Improves performance and prevents UI glitches.
-

◆ What Happens if Keys Are Not Unique?

If keys are duplicated or unstable:

1. Incorrect UI updates – React may reuse the wrong component instance.
 - o Example: input values resetting when list changes.
2. Performance loss – React may re-render unnecessarily.
3. Bugs in stateful components – A component might keep the wrong state because React thinks it's the same item.

Example Problem:

```
const items = ["Apple", "Banana", "Cherry"];
```

```
function MyList() {  
  return (  
    <ul>
```

```

{items.map((item, index) => (
  <li key={index}>{item}</li> // ❌ Bad: using index
))}
</ul>
);
}

```

👉 If we reorder items (["Banana", "Apple", "Cherry"]), React thinks only the text changed, not the positions. This causes wrong component re-use.

♦ Best Practices for Keys

✅ Use a unique, stable identifier (like id from DB).

✅ Use index only when:

- List is static (no reordering/deletion).
- Items don't have unique IDs.

❌ Don't use random values (e.g., Math.random()), as they change on every render.

How do custom hooks work and what are their benefits?

// → Have you created a reusable hook in a project?

What are Custom Hooks?

- A custom hook is a JavaScript function whose name starts with use and can call other hooks (useState, useEffect, etc.).
- They let you extract reusable logic from components so it can be shared across multiple components.

Example: A simple custom hook

```
import { useState, useEffect } from "react";
```

```

function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);
}

```

```
    return width;
}
```

// Usage in a component

```
function MyComponent() {
  const width = useWindowWidth();
  return <p>Window width: {width}px</p>;
}
```

✅ Here:

- The logic for tracking window width is encapsulated in useWindowWidth.
- Any component can reuse it without rewriting the logic.

◆ Benefits of Custom Hooks

1. Reusability – Write logic once, use it anywhere.
2. Separation of concerns – Keeps components clean; hooks handle the logic.
3. Testability – Easier to test logic separately from UI.
4. Readability & maintainability – Avoid repeating code in multiple components.

◆ Common Use Cases

- API calls / data fetching
- Form input handling and validation
- Local storage management
- Window size or scroll position tracking
- Authentication/session management

◆ Real-world Example (from a project)

Suppose I built a finance tracker app:

- I created a custom hook useFetchData(url) to fetch transactions from API:

```
function useFetchData(url) {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);
```

```

useEffect(() => {
  async function fetchData() {
    setLoading(true);
    const res = await fetch(url);
    const result = await res.json();
    setData(result);
    setLoading(false);
  }
  fetchData();
}, [url]);

return { data, loading };
}

```

// Usage

```

function Transactions() {
  const { data, loading } = useFetchData("/api/transactions");
  if (loading) return <p>Loading...</p>;
  return <ul>{data.map(t => <li key={t.id}>{t.amount}</li>)}</ul>;
}

```

- Benefit: Any component that needs API data can just use useFetchData(url) without repeating the fetching logic.

What is React's Concurrent Mode and how does it improve UX?

// → What are transitions and how do they relate to concurrent features?

1. What is Concurrent Mode?

- Concurrent Mode is a set of React features (React 18+) that allows React to interrupt, pause, or prioritize work without blocking the main thread.
- Goal: Make apps feel more responsive, especially under heavy updates.

Key Idea:

- Traditional React is synchronous: once rendering starts, it blocks the UI until complete.
 - Concurrent Mode is asynchronous: rendering can be paused, delayed, or interrupted for higher-priority tasks (like user input).
-

◆ 2. How Concurrent Mode Improves UX

- **Non-blocking rendering:** User interactions (clicks, typing) stay responsive even while React updates large UI trees.
 - **Smooth transitions:** React can delay non-urgent updates without freezing the interface.
 - **Avoids jank:** Heavy computations or data fetching won't block the UI.
-

◆ 3. Transitions

- Transitions are a new API in Concurrent Mode to mark certain state updates as “non-urgent”.
- Example: Updating a search results list while keeping the search input responsive.

API:

```
const [search, setSearch] = useState("");  
const [results, setResults] = useState([]);  
const [isPending, startTransition] = useTransition();
```

```
function handleChange(e) {  
  setSearch(e.target.value); // urgent update (input)  
  startTransition(() => {  
    // non-urgent update (results list)  
    setResults(filterData(e.target.value));  
  });  
}
```

How it works:

1. `setSearch` → urgent → input updates immediately.
2. `startTransition` → non-urgent → list update can be deferred if the browser is busy.
3. React keeps the UI interactive while rendering the heavy part in the background.

✅ You can also use `isPending` to show a loading indicator during non-urgent updates.

◆ 4. Relation Between Concurrent Mode and Transitions

Feature	Role in Concurrent Mode
---------	-------------------------

Concurrent Mode	Makes React rendering interruptible & non-blocking
-----------------	--

Transitions	Marks certain updates as low-priority to avoid UI blocking
-------------	--

- Together, they allow responsive apps even under heavy computation or large UI trees.
-

◆ 5. Example Use Cases

- Search/filter UI: input stays responsive while results update.
- Pagination or virtualized lists: scrolling stays smooth.
- Animations: can render complex UI without jank.
- Large forms: typing isn't blocked while validating or saving data.

What is hydration in SSR and how does it work with React?

// → What issues have you faced with SSR/Next.js?

1. What is Hydration?

- Hydration is the process where a server-rendered HTML page is “attached” to the React app on the client so that it becomes interactive.
- SSR sends fully rendered HTML to the browser, but without JavaScript, it's static content.
- React then hydrates the DOM, attaches event listeners, and makes the page dynamic.

How it works:

1. Server-side: React renders components to HTML and sends it to the browser.
 2. `<button>Click me</button> <!-- HTML already rendered -->`
 3. Client-side (hydration): React runs JS, attaches event listeners, and manages state.
 4. `const [count, setCount] = useState(0); // now interactive`
 5. `<button onClick={() => setCount(count + 1)}>Click me</button>`
- After hydration, the page behaves like a normal React SPA.

◆ 2. Why Hydration is Important

- Improves initial page load → faster content for SEO and users.
- Makes the page interactive without re-rendering the entire DOM.
- Allows SSR + CSR hybrid: static content for speed, React for interactivity.

◆ 3. Common Issues Faced with SSR / Hydration

1. Mismatch Between Server & Client

- React compares the server HTML and client-rendered HTML.
- If they differ, you get warnings:
- Warning: Text content did not match. Server: "X" Client: "Y"
- Causes:
 - o Using browser-specific APIs (window, document) on the server.

- o Random values (Math.random, Date.now) during render.
- o Non-deterministic rendering.

2. Event Listeners Not Attached

- SSR renders HTML, but without hydration, buttons/inputs aren't interactive.

3. Performance Overhead

- Large pages may take time to hydrate, causing "interactive delay".
- Heavy JS bundles can slow down hydration.

4. State Initialization Issues

- Server has no access to client state (localStorage, sessionStorage).
- You may need useEffect to initialize state after hydration.

◆ 4. How to Handle These Issues

1. Conditional client-only code:
2. `useEffect(() => {`
3. `console.log(window.innerWidth);`
4. `}, []);`
5. Avoid non-deterministic server-side rendering (e.g., no Math.random() in render).
6. Use Next.js dynamic imports for client-only components:
7. `import dynamic from 'next/dynamic';`
8. `const Chart = dynamic(() => import('./Chart'), { ssr: false });`
9. Split hydration: defer heavy components using React.lazy or Suspense.
10. Check hydration warnings: make sure server & client markup match exactly.

How does error handling work in React with Error Boundaries?

// → What types of errors can and can't be caught?

1. What Are Error Boundaries?

- Error boundaries are React components that catch JavaScript errors anywhere in their child component tree.
- They prevent the whole React component tree from crashing and let you show a fallback UI.

How to create one (class component):

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
```



```

    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    // Update state to show fallback UI
    return { hasError: true };
  }

  componentDidCatch(error, info) {
    // Log error to an external service
    console.log(error, info);
  }

  render() {
    if (this.state.hasError) {
      return <h2>Something went wrong.</h2>;
    }
    return this.props.children;
  }
}

// Usage
function App() {
  return (
    <ErrorBoundary>
      <MyComponent />
    </ErrorBoundary>
  );
}

```

◆ 2. What Types of Errors Can Be Caught?

Error boundaries can catch errors in:

1. Render phase – errors during rendering of child components.

2. Lifecycle methods – errors in `componentDidMount`, `componentDidUpdate`.
 3. Constructor of child components – errors in child class component constructors.
-

◆ 3. What Cannot Be Caught?

Error boundaries cannot catch:

1. Event handlers – handle separately with `try/catch`.
 2. `<button onClick={() => { throw new Error("Oops") }}>Click</button>`
 - o React does not crash the whole app; the error must be handled manually.
 3. Asynchronous code – e.g., `setTimeout`, fetch promises.
 4. Server-side rendering errors – errors during SSR must be handled differently.
 5. Errors in the error boundary itself – must wrap multiple boundaries if needed.
-

◆ 4. Using Hooks with Error Boundaries

- You cannot use a functional component with hooks alone as an error boundary.
 - Only class components can be error boundaries.
 - Workarounds: use a wrapper class boundary to catch errors for functional children.
-

◆ 5. Best Practices

- Place error boundaries around sections of the app, not just the root.
- Use fallback UI that allows the app to continue running.
- Log errors to monitoring services (Sentry, LogRocket).
- Combine with `try/catch` in async operations or event handlers.

What is the difference between `useRef` and `createRef`?

// → When should refs be avoided altogether?

1. `useRef` (Hook, functional components)

- Purpose: Persist a mutable value across renders without triggering re-renders.
- Scope: Works inside functional components.
- Returns an object: `{ current: ... }`

Example:

```
import { useRef, useEffect } from "react";
```

```
function Timer() {
  const intervalRef = useRef(null);

  useEffect(() => {
    intervalRef.current = setInterval(() => console.log("tick"), 1000);
    return () => clearInterval(intervalRef.current);
  }, []);

  return <p>Timer running in console</p>;
}
```

✅ Key points:

- intervalRef.current persists across renders.
- Updating current does not cause a re-render.

◆ 2. createRef (Class components)

- Purpose: Access a DOM node or a React component instance.
- Scope: Typically used in class components.
- Returns a new ref on each render, so it's usually attached to instance variables in classes.

Example:

```
class MyInput extends React.Component {
  constructor(props) {
    super(props);
    this.inputRef = React.createRef();
  }

  focusInput = () => {
    this.inputRef.current.focus();
  }

  render() {
    return <input ref={this.inputRef} />;
  }
}
```

✓ Key points:

- createRef does not persist across functional component renders.
- Works best in class components.

◆ 3. Key Differences

Feature	useRef (functional)	createRef (class)
Component type	Functional	Class
Persistence	Same ref across renders	New ref every render
Updates trigger rerender?	✗ No	✗ No
Common use cases	DOM access, timers, mutable values	DOM access, child component instance

◆ 4. When to Avoid Refs

Refs should be used sparingly. Prefer state/props instead. Avoid refs when:

1. Controlling UI state – e.g., form inputs should use controlled components, not refs.
2. Conditional rendering – rely on state, not refs, for showing/hiding components.
3. Complex logic – keep data in state to leverage React's reconciliation and re-rendering.

✓ Rule of Thumb: "Refs are escape hatches, not replacements for state."

◆ 5. When Refs Are Useful

- Accessing DOM elements (focus, scroll, measure).
- Storing mutable values that don't trigger re-renders (timers, intervals, previous state).
- Integrating with third-party libraries that require DOM nodes.

How does React handle synthetic events under the hood?

// → What are the performance benefits of the synthetic event system?

1. What Are Synthetic Events?

- React wraps native browser events in its own SyntheticEvent object.
- This provides a consistent API across all browsers (cross-browser compatibility).
- Works the same for all events: click, change, keypress, etc.

```
function Button() {  
  const handleClick = (event) => {  
    console.log(event.type); // "click"  
  };  
}
```

```
return <button onClick={handleClick}>Click Me</button>;  
}
```

- Here, event is a SyntheticEvent, not the raw DOM event.
-

◆ 2. How It Works Under the Hood

1. Single Event Listener

- o React attaches a single event listener at the root of the DOM (event delegation).
- o Example: React sets one listener on document instead of adding one per element.

2. Event Pooling (in older versions)

- o Synthetic events were pooled for memory efficiency (React reuses event objects).
- o In React 17+, pooling is removed, but the synthetic wrapper still exists.

3. Propagation Control

- o React normalizes event propagation (stopPropagation, preventDefault) across browsers.
-

◆ 3. Performance Benefits

1. Memory Efficiency

- o Instead of attaching many listeners for every component, React uses one listener per event type at the root → less memory.

2. Faster Event Handling

- o Centralized delegation avoids multiple DOM lookups.
- o Updates can be batched efficiently during the event.

3. Cross-Browser Consistency

- o You don't have to worry about differences in addEventListener, event bubbling, or event properties.

4. Simplified Cleanup

- o No need to manually remove listeners in most cases; React handles it automatically when components unmount.
-

◆ 4. Things to Note

- Synthetic events are pooled in older versions, so you must call event.persist() if you need to access them asynchronously.
- React's synthetic events still use the native events under the hood; the wrapper just normalizes them.

- Works seamlessly with hooks, state updates, and batching in modern React.

You have a large form with many inputs. how would you manage and optimize state?

1. Controlled vs Uncontrolled Inputs

- Controlled inputs: state lives in React → easy validation, dynamic UI updates.
- Uncontrolled inputs: state lives in DOM → less re-rendering, simpler for static forms.

Recommendation: For large, dynamic forms, use controlled components for critical fields and uncontrolled for rarely updated fields.

◆ 2. State Management Strategies

a) Single State Object

- Store all form values in a single object using useState:

```
const [formData, setFormData] = useState({
  firstName: "",
  lastName: "",
  email: "",
  // ...more fields
});

function handleChange(e) {
  const { name, value } = e.target;
  setFormData(prev => ({ ...prev, [name]: value }));
}
```

✅ Pros:

- Easy to submit the entire form.
- Centralized state.

⚠️ Cons:

- Every change re-renders the entire form, which can hurt performance for large forms.

b) Split State per Input / Field

```
const [firstName, setFirstName] = useState("");
const [lastName, setLastName] = useState("");
```

✅ Pros:

- Updates only the relevant input → fewer re-renders.

⚠ Cons:

- Becomes verbose with many fields.
-

c) useReducer

- For complex forms with nested fields or dynamic input arrays:

```
function reducer(state, action) {  
  switch(action.type) {  
    case "UPDATE_FIELD":  
      return { ...state, [action.field]: action.value };  
    default: return state;  
  }  
}
```

```
const [formState, dispatch] = useReducer(reducer, initialState);
```

```
<input  
  name="firstName"  
  value={formState.firstName}  
  onChange={(e) => dispatch({ type: "UPDATE_FIELD", field: e.target.name, value: e.target.value })}  
>
```

✅ Pros:

- Centralized logic for validation, updates, and resets.
 - Great for dynamic/nested forms.
-

◆ 3. Performance Optimization

1. React.memo

- Wrap form sections or input components with React.memo to prevent unnecessary re-renders.

2. useCallback

- Memoize handlers to prevent child re-rendering:

```
const handleChange = useCallback((e) => {  
  dispatch({ type: "UPDATE_FIELD", field: e.target.name, value: e.target.value });  
}, []);
```

3. Lazy Rendering / Virtualization

- For very long forms (100+ fields), render only visible fields, similar to virtualized lists.

4. Debounce Expensive Computations

- For live validation, API calls, or formatting, use debounce to reduce frequency of updates.

◆ 4. Libraries That Help

For very large forms, consider libraries designed for performance and scalability:

- Formik – popular and battle-tested.
- React Hook Form – minimal re-renders, very performant for large forms.
- Final Form – provides advanced control over state and validation.

Example: React Hook Form only re-renders inputs that change, making it ideal for large forms.

◆ 5. Best Practices Summary

1. Use controlled components for important inputs.
2. Group fields using useReducer or libraries like React Hook Form.
3. Optimize re-renders with React.memo and useCallback.
4. Avoid updating state unnecessarily (don't copy entire state if only one field changes).
5. Use lazy rendering or virtualization for extremely large forms.

You're seeing unnecessary re-renders in a component how do you debug and fix it?

1. Identify the Problem

a) React DevTools Profiler

- Open React DevTools → Profiler.
- Record interactions and check which components are re-rendering frequently.
- Look for:
 - o Components updating even when props/state haven't changed.
 - o Large components re-rendering unnecessarily.

b) Logging Render

```
function MyComponent(props) {  
  console.log("MyComponent rendered");  
  return <div>{props.value}</div>;  
}
```


- Logs in console help identify how often a component renders.
 - Combine with Profiler for more insights.
-

♦ 2. Common Causes of Unnecessary Re-renders

1. Parent Re-rendering
 - o Even if the child's props didn't change, it re-renders by default.
 2. Inline Functions / Objects / Arrays
 3. `<Child onClick={() => doSomething()} />` // new function every render
 4. `<Child data={{ a: 1 }} />` // new object every render
 5. State updates that don't actually change state
 6. `setCount(count);` // triggers a re-render even if value didn't change
 7. Context updates
 - o Any change in context value triggers all consumers to re-render.
-

♦ 3. How to Fix / Optimize

a) React.memo for Functional Components

- Memoize the component to skip re-render if props didn't change.

```
const Child = React.memo(function Child({ value }) {  
  console.log("Child rendered");  
  return <div>{value}</div>;  
});
```

b) useCallback for Functions

- Memoize functions passed as props to avoid new references on every render.

```
const handleClick = useCallback(() => doSomething(), []);  
<Child onClick={handleClick} />
```

c) useMemo for Expensive Calculations

- Memoize derived data so it doesn't recompute every render.

```
const processedData = useMemo(() => expensiveCalculation(data), [data]);
```

d) Split State / Lift State Wisely

- Avoid putting unrelated state in one parent.
- Split state to local components to minimize re-renders.

e) Optimize Context Usage

- Only wrap components that need context.

- Consider splitting context for different pieces of state.

f) Avoid Inline Objects / Arrays in JSX

// Bad

```
<Child data={{ a: 1 }} />
```

// Good

```
const data = useMemo(() => ({ a: 1 }), []);
```

```
<Child data={data} />
```

◆ 4. Tools to Aid Debugging

- React DevTools Profiler – visualize re-renders.
- why-did-you-render – NPM package that logs unnecessary re-renders in development.
- console.log – simple but effective for small components.

◆ 5. Best Practices Summary

1. Use React.memo for pure functional components.
2. Memoize functions and objects passed as props with useCallback / useMemo.
3. Avoid putting unrelated state in the same component.
4. Minimize context consumers to only components that need them.
5. Use Profiler to measure and verify performance improvements.

A parent component re-renders and causes all children to re-render. how can you optimize?

1. Why Children Re-render

- In React, when a parent re-renders, all its children re-render by default.
- Even if child props haven't changed, the child function runs again.
- Causes unnecessary computations and can hurt performance in large trees.

◆ 2. Optimization Strategies

a) React.memo

- Wrap child components with React.memo to prevent re-render if props didn't change.

```
const Child = React.memo(({ value }) => {  
  console.log("Child rendered");  
  return <div>{value}</div>;  
})
```

```
});
```

```
function Parent() {  
  const [count, setCount] = useState(0);  
  return (  
    <>  
      <button onClick={() => setCount(count + 1)}>Increment</button>  
      <Child value="Static Data" />  
    </>  
  );  
}
```

✅ Now Child only re-renders if value changes.

b) useCallback for Functions

- If a parent passes inline functions as props, they are recreated each render → child re-renders.

```
const handleClick = useCallback(() => doSomething(), []);
```

```
<Child onClick={handleClick} />
```

- useCallback ensures the function reference stays stable.
-

c) useMemo for Objects / Arrays

- Passing inline objects/arrays also triggers child re-renders.

```
// Bad
```

```
<Child data={{ a: 1 }} />
```

```
// Good
```

```
const data = useMemo(() => ({ a: 1 }), []);
```

```
<Child data={data} />
```

d) Split State

- Keep state local to the component that needs it instead of storing it in the parent.
- This reduces the frequency of parent re-renders.

```
function Parent() {  
  const [count, setCount] = useState(0);
```

```
return <Child />;  
}
```

- Child won't re-render if it doesn't depend on count.

e) Optimize Context Usage

- If a parent re-renders due to context updates, all context consumers re-render.
- Split context into multiple providers for unrelated pieces of state.

f) Lazy / Dynamic Rendering

- For large child lists, consider:
 - o React.lazy / Suspense for dynamic imports.
 - o Windowing / virtualization for long lists (react-window, react-virtualized).

◆ 3. Summary of Techniques

Problem Cause	Solution
Child re-renders with same props	React.memo
Inline functions as props	useCallback
Inline objects/arrays as props	useMemo
Parent state unrelated to child	Split state / localize
Context triggers large re-renders	Split context providers
Heavy component trees	Lazy load / virtualization

You need to share data across deeply nested components. how do you approach it?

1. Prop Drilling

- Passing props from parent → child → grandchild repeatedly.
- Works for small trees or few nested levels.

```
function Parent() {  
  const user = { name: "Sai" };  
  return <Child user={user} />;  
}
```

```
function Child({ user }) {  
  return <GrandChild user={user} />;  
}
```

```
}
```

```
function GrandChild({ user }) {  
  return <p>{user.name}</p>;  
}
```

✅ Simple for small apps

❌ Becomes tedious and error-prone with many levels → “prop drilling problem”

◆ 2. React Context API

- Avoids prop drilling by creating a global-ish data store accessible to all nested components.

Example:

```
const UserContext = React.createContext();
```

```
function App() {  
  const user = { name: "Sai" };  
  return (  
    <UserContext.Provider value={user}>  
      <Parent />  
    </UserContext.Provider>  
  );  
}
```

```
function GrandChild() {  
  const user = React.useContext(UserContext);  
  return <p>{user.name}</p>;  
}
```

✅ Good for theming, auth, language, global settings

⚠️ Frequent context updates re-render all consumers, so split context if needed

◆ 3. State Management Libraries

For large-scale apps, Context may become insufficient. Libraries provide more control, especially for frequent updates or complex data flows.

- Redux / Redux Toolkit – centralized store, predictable state, middlewares for async.

- Zustand – simple, minimal re-renders, modern alternative.
- MobX – observable state, auto-reactive components.

Example with Redux:

```
// Store
```

```
const initialState = { user: { name: "Sai" } };
```

```
// Component
```

```
const name = useSelector(state => state.user.name);
```

- ✅ Works well for large apps, easy debugging and devtools integration

◆ 4. Other Patterns

1. Custom Hooks – encapsulate logic and state to share across multiple components.

```
function useUser() {
  const [user, setUser] = useState({ name: "Sai" });
  return { user, setUser };
}
```

- Can be combined with Context for global state.
2. Render Props / HOCs – less common now, mostly replaced by hooks.

◆ 5. Best Practices

- Use props for small, local data passing.
- Use Context for static or infrequently updated global data (theme, auth).
- Use state management libraries for frequently updated or complex data.
- Split contexts or stores to minimize unnecessary re-renders.

How do you ensure accessibility (a11y) in your components?

1. Use Semantic HTML

- Use elements according to their meaning:
 - o <button> instead of <div> for clickable actions
 - o <header>, <main>, <footer> for layout
 - o <label> for form inputs
- Semantic HTML gives screen readers context automatically.

```
<label htmlFor="email">Email</label>
```

```
<input id="email" type="email" />
```

◆ 2. ARIA Attributes

- ARIA (Accessible Rich Internet Applications) attributes provide extra info when semantic HTML isn't enough.
- Common attributes:
 - o `aria-label`, `aria-labelledby` → label for elements
 - o `aria-hidden` → hide elements from screen readers
 - o `role` → define element role when semantic HTML isn't used

`<button aria-label="Close menu">X</button>`

✅ Use ARIA only when necessary; prefer semantic HTML first.

◆ 3. Keyboard Accessibility

- Ensure all interactive elements are focusable with Tab.
- Use proper keyboard events (Enter, Space) for actions.
- Avoid using `div` or `span` as buttons unless you handle keyboard interactions.

```
<div
  role="button"
  tabIndex={0}
  onKeyDown={(e) => e.key === 'Enter' && doSomething()}
  onClick={doSomething}
>
  Click me
</div>
```

- Better: just use `<button>`.

◆ 4. Focus Management

- Manage focus for modals, dialogs, and dynamic content:
 - o Trap focus inside modals (`react-focus-lock`)
 - o Return focus to the triggering element when closing modal

// Example: `useRef` to focus input on modal open

```
const inputRef = useRef(null);
useEffect(() => { inputRef.current.focus(); }, []);
```

◆ 5. Color Contrast & Visual Cues

- Ensure text contrast ratio $\geq 4.5:1$ for readability.
 - Avoid color-only indicators; use icons or text alongside colors.
-

◆ 6. Testing Accessibility

- Automated tools:
 - o `eslint-plugin-jsx-a11y` → linting rules for React JSX
 - o `axe-core` → accessibility testing
- Manual testing:
 - o Keyboard navigation
 - o Screen reader testing (VoiceOver, NVDA)

`npm install eslint-plugin-jsx-a11y --save-dev`

◆ 7. Best Practices in React

- Prefer controlled form components with `<label>` for accessibility.
- Avoid auto-focusing elements unnecessarily; it can confuse screen readers.
- Provide alt text for images and descriptive links.
- Use skip links (`<a href="#main">Skip to content</a>`) for keyboard users.

You're asked to migrate a class-based component to a functional one with hooks. what are the steps?

1. Identify the Component Structure

- Take note of the class component's:
 - o State (`this.state`)
 - o Lifecycle methods (`componentDidMount`, `componentDidUpdate`, `componentWillUnmount`)
 - o Event handlers
 - o Props usage

Example class component:

```
class Counter extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = { count: 0 };  
  }  
}
```



```
componentDidMount() {  
  console.log("Mounted");  
}
```

```
componentDidUpdate() {  
  console.log("Updated");  
}
```

```
componentWillUnmount() {  
  console.log("Unmounting");  
}
```

```
increment = () => {  
  this.setState({ count: this.state.count + 1 });  
};
```

```
render() {  
  return (  
    <div>  
      <p>{this.state.count}</p>  
      <button onClick={this.increment}>Increment</button>  
    </div>  
  );  
}
```

◆ 2. Convert to Functional Component

- Remove the class keyword and render() method.
- Create a function that takes props.

```
function Counter(props) {  
  // logic goes here  
  return (  
    <div>
```

```
    {/* JSX */}  
  </div>  
);  
}
```

◆ 3. Convert State to useState

- Replace this.state and this.setState with useState.

```
const [count, setCount] = useState(0);
```

```
const increment = () => setCount(count + 1);
```

◆ 4. Replace Lifecycle Methods with useEffect

Class Lifecycle	Functional Hook Equivalent
componentDidMount	useEffect(() => { ... }, [])
componentDidUpdate	useEffect(() => { ... }, [dependencies])
componentWillUnmount	useEffect(() => { return () => { ... } }, [])

Example:

```
useEffect(() => {  
  console.log("Mounted or updated"); // combine mount/update logic  
  return () => console.log("Unmounting"); // cleanup  
}, [count]); // dependency array
```

◆ 5. Convert Event Handlers

- Remove this references.
- Use arrow functions or defined functions inside the component.

```
const increment = () => setCount(count + 1);  
<button onClick={increment}>Increment</button>
```

◆ 6. Props Handling

- this.props → directly use props argument.

```
function Counter({ initialCount }) {  
  const [count, setCount] = useState(initialCount);  
}
```

◆ 7. Test Component Behavior

- Verify:
 - o State updates correctly
 - o Lifecycle logic works (mounting, updating, cleanup)
 - o Event handlers function as expected
-

◆ 8. Optional: Optimize with Hooks

- If needed, add:
 - o `useMemo` for expensive computations
 - o `useCallback` for memoized handlers
 - o `useRef` for DOM references
-

✅ Final Functional Component Example

```
function Counter({ initialCount = 0 }) {  
  const [count, setCount] = useState(initialCount);  
  
  useEffect(() => {  
    console.log("Mounted or updated");  
    return () => console.log("Unmounting");  
  }, [count]);  
  
  const increment = () => setCount(count + 1);  
  
  return (  
    <div>  
      <p>{count}</p>  
      <button onClick={increment}>Increment</button>  
    </div>  
  );  
}
```

Summary Steps:

1. Convert class → function.
2. Replace state with useState.
3. Replace lifecycle methods with useEffect.
4. Remove this references in props, state, and methods.
5. Update event handlers.
6. Test for correct behavior.
7. Optionally optimize with useMemo, useCallback, or useRef.

An API fetch needs to be cancelled if the user navigates away.how would you implement that?

1. Using AbortController with fetch

```
import { useEffect, useState } from "react";
```

```
function UserList() {
```

```
  const [users, setUsers] = useState([]);
```

```
  const [loading, setLoading] = useState(true);
```

```
  useEffect(() => {
```

```
    const controller = new AbortController(); // create a controller
```

```
    const signal = controller.signal;
```

```
    async function fetchUsers() {
```

```
      try {
```

```
        const response = await fetch("https://api.example.com/users", { signal });
```

```
        const data = await response.json();
```

```
        setUsers(data);
```

```
        setLoading(false);
```

```
      } catch (error) {
```

```
        if (error.name === "AbortError") {
```

```
          console.log("Fetch aborted");
```

```
        } else {
```

```
          console.error("Fetch error:", error);
```

```
        }
```

```
      }
```

```

}

fetchUsers();

// Cleanup: cancel fetch if component unmounts
return () => controller.abort();
}, []); // empty dependency → runs once on mount

if (loading) return <p>Loading...</p>;
return (
  <ul>
    {users.map(user => <li key={user.id}>{user.name}</li>)}
  </ul>
);
}

```

◆ 2. Key Points

1. `AbortController` is a native JS API for canceling fetch requests.
 2. The `signal` option links the fetch to the controller.
 3. In the `useEffect` cleanup function, calling `controller.abort()` cancels the fetch if the component unmounts.
 4. Always handle the `"AbortError"` to avoid logging unnecessary errors.
-

◆ 3. With Axios

If you use `Axios` instead of `fetch`, it has a `CancelToken` (Axios < v1) or `AbortController` (Axios v1+).

```

import axios from "axios";
import { useEffect, useState } from "react";

function UserList() {
  const [users, setUsers] = useState([]);
  useEffect(() => {
    const controller = new AbortController();

```

```

    axios.get("https://api.example.com/users", { signal: controller.signal })
      .then(res => setUsers(res.data))
      .catch(err => {
        if (axios.isCancel(err)) console.log("Request cancelled");
        else console.error(err);
      });

    return () => controller.abort();
  }, []);

  return <div>{users.length} users loaded</div>;
}

```

◆ 4. Best Practices

- Always cleanup async operations in useEffect.
- Avoid setting state on unmounted components.
- For multiple API calls, consider centralized abort controllers.
- If using libraries like React Query or SWR, they handle automatic cancellation for you.

You need to make a component that is reused in multiple projects. how do you design it?

1. Identify the Core Functionality

- Determine what the component must do versus what can be customizable.
 - Example: A Button component must handle clicks, display a label, and support different sizes and colors.
-

◆ 2. Design a Clear API (Props)

- Expose props for customization instead of hardcoding values.
- Props can control:
 - o Content → children, label
 - o Styling → className, style, variant, size
 - o Behavior → onClick, disabled, loading
- Keep default values for common use cases.

```
function Button({ label, onClick, size = "medium", variant = "primary", disabled = false }) {
```

```

return (
  <button
    onClick={onClick}
    disabled={disabled}
    className={`btn ${variant} ${size}`}
  >
    {label}
  </button>
);
}

```

◆ 3. Styling Flexibility

- Avoid hard-coded styles. Use:
 - o CSS Modules / Tailwind / styled-components
 - o Allow className or style overrides for custom styling.

```
<button className={`btn ${variant} ${size} ${className}`} />
```

- Enables project-specific styling without changing the component code.
-

◆ 4. Accessibility (a11y)

- Include semantic HTML and ARIA attributes by default.
 - Example for a Button:
 - o Ensure keyboard accessibility
 - o Handle disabled and aria-label
-

◆ 5. Handle Variability & Extensibility

- Make the component composable: support custom children or render props if needed.

```
<Button>
```

```
<span>🚀 Launch</span>
```

```
</Button>
```

- Use slots, render props, or children for advanced customization.
-

◆ 6. Avoid Project-Specific Dependencies

- Component should be agnostic of a particular project.

- Don't import project-specific state, context, or assets.
 - Accept everything configurable via props.
-

◆ 7. Testing

- Write unit tests for:
 - o Rendering with different props
 - o Event handling (onClick)
 - o Accessibility (aria-* attributes)
-

◆ 8. Documentation

- Provide clear prop types and usage examples.
- Use:
 - o propTypes or TypeScript interfaces
 - o Storybook or a demo page for visualization

```
interface ButtonProps {  
  label: string;  
  onClick: () => void;  
  size?: "small" | "medium" | "large";  
  variant?: "primary" | "secondary";  
  disabled?: boolean;  
}
```

◆ 9. Packaging for Reuse

- If sharing across projects:
 - o Publish as an npm package
 - o Or use a monorepo (e.g., Turborepo, Nx)
 - o Ensure versioning, changelog, and dependencies are clear
-

◆ 10. Best Practices Summary

1. Keep API simple but flexible.
2. Make styling customizable via className or theme props.
3. Ensure accessibility by default.
4. Keep it decoupled from specific projects.

5. Support composition and advanced use cases.
6. Write tests and documentation.
7. Package for easy integration.

How do you secure a React frontend in terms of handling tokens and sensitive info?

1. Don't Store Sensitive Secrets in the Frontend

- Never hardcode API keys, secrets, or credentials in your React code.
 - All sensitive logic should live on the server.
 - Frontend can only store temporary tokens like JWT for authentication.
-

◆ 2. Handling Tokens Securely

a) Prefer HttpOnly Cookies over localStorage/sessionStorage

- HttpOnly cookies cannot be accessed via JavaScript → protects against XSS attacks.
- Example: backend sets cookie with Set-Cookie: HttpOnly; Secure; SameSite=Strict.

b) If Using localStorage / sessionStorage

- Only for short-lived tokens or non-critical data.
- Always validate tokens on the backend.

c) Token Expiration & Refresh

- Use short-lived access tokens + refresh tokens.
 - Automatically refresh tokens via silent API calls, not manual input.
-

◆ 3. Protect Against XSS (Cross-Site Scripting)

- Sanitize user input before rendering.
- Avoid dangerouslySetInnerHTML unless necessary and sanitized.
- Use libraries like DOMPurify if rendering HTML from external sources.

import DOMPurify from 'dompurify';

<p dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(userInput) }} />

◆ 4. Secure API Requests

- Use HTTPS for all API requests.
- Add CORS restrictions on the backend.
- Don't rely on frontend-only security; always validate tokens server-side.

```

fetch('/api/data', {
  method: 'GET',
  headers: {
    'Authorization': `Bearer ${accessToken}`,
  },
});

```

◆ 5. Avoid Storing Sensitive Data in Memory for Long

- Keep tokens in state or memory while needed, and clear on logout.
 - Don't leave sensitive info in global variables or browser storage longer than necessary.
-

◆ 6. Secure Routing

- Protect routes using authentication guards.
- Example with React Router:

```

function PrivateRoute({ children }) {
  const isAuthenticated = Boolean(localStorage.getItem("token"));
  return isAuthenticated ? children : <Navigate to="/login" />;
}

```

- Always verify auth on backend, not just frontend.
-

◆ 7. Miscellaneous Best Practices

1. Content Security Policy (CSP) – prevents malicious scripts from executing.
 2. Subresource Integrity (SRI) – validate external scripts.
 3. Environment Variables – use .env for config, never commit secrets.
 4. Secure dependencies – keep packages up to date and scan for vulnerabilities.
-

◆ 8. Summary Table

Risk / Requirement	Best Practice
Token storage	HttpOnly cookies preferred, avoid localStorage for sensitive data
XSS protection	Sanitize input, avoid dangerouslySetInnerHTML
API communication	HTTPS + backend validation + CORS
Routing / access control	Frontend guards + backend validation
Secrets	Never hardcode, use environment variables

// Machine Coding

// *Build a counter app with increment, decrement and reset buttons.

```
import React, { useState } from "react";
```

```
function CounterApp() {
```

```
  const [count, setCount] = useState(0);
```

```
  const increment = () => setCount(prevCount => prevCount + 1);
```

```
  const decrement = () => setCount(prevCount => prevCount > 0 ? prevCount - 1 : 0);
```

```
  const reset = () => setCount(0);
```

```
  return (
```

```
    <div style={styles.container}>
```

```
      <h1>Counter App</h1>
```

```
      <p style={styles.count}>{count}</p>
```

```
      <div style={styles.buttonContainer}>
```

```
        <button onClick={increment} style={styles.button}>Increment</button>
```

```
        <button disabled={count === 0} onClick={decrement}  
style={styles.button}>Decrement</button>
```

```
        <button onClick={reset} style={styles.button}>Reset</button>
```

```
      </div>
```

```
    </div>
```

```
);  
}
```

```
// Inline styles
```

```
const styles = {  
  container: {  
    display: "flex",  
    flexDirection: "column",  
    alignItems: "center",  
    marginTop: "50px",  
    fontFamily: "Arial, sans-serif",  
  },  
  count: {  
    fontSize: "48px",  
    margin: "20px 0",  
  },  
  buttonContainer: {  
    display: "flex",  
    gap: "10px",  
  },  
  button: {  
    padding: "10px 20px",  
    fontSize: "16px",  
    cursor: "pointer",  
  },  
};
```

```
export default CounterApp;
```

```
{/* ----- */}
```

```
{/* *create a crud operation - name, gender option, state, city dropdown,  
pincode validation, email validation, mobile number validation, Choose  
Department dropdown, salary currency field, and end of form confirm  
box */}
```

```
import React, { useState, useEffect } from "react";

// Sample states and cities
const stateCityData = {
  California: ["Los Angeles", "San Francisco", "San Diego"],
  Texas: ["Houston", "Dallas", "Austin"],
  Florida: ["Miami", "Orlando", "Tampa"],
};

// Sample departments
const departments = ["Engineering", "HR", "Marketing", "Sales"];

function CrudForm() {
  const initialState = {
    id: null,
    name: "",
    gender: "",
    state: "",
    city: "",
    pincode: "",
    email: "",
    mobile: "",
    department: "",
    salary: "",
    confirm: false,
  };

  const [formData, setFormData] = useState(initialFormState);
  const [entries, setEntries] = useState([]);
  const [cities, setCities] = useState([]);
  const [editId, setEditId] = useState(null);
  const [errors, setErrors] = useState({});
```

```

// Update cities when state changes
useEffect(() => {
  if (formData.state) setCities(stateCityData[formData.state] || []);
  else setCities([]);
  setFormData({ ...formData, city: "" }); // reset city
}, [formData.state]);

// Handle input change
const handleChange = (e) => {
  const { name, value, type, checked } = e.target;
  setFormData({ ...formData, [name]: type === "checkbox" ? checked : value });
};

// Validate form
const validate = () => {
  const newErrors = {};
  if (!formData.name) newErrors.name = "Name is required";
  if (!formData.gender) newErrors.gender = "Gender is required";
  if (!formData.state) newErrors.state = "State is required";
  if (!formData.city) newErrors.city = "City is required";
  if (!/^d{5,6}$/.test(formData.pincodes)) newErrors.pincodes = "Invalid pincodes";
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(formData.email)) newErrors.email = "Invalid email";
  if (!/^d{10}$/.test(formData.mobile)) newErrors.mobile = "Invalid mobile number";
  if (!formData.department) newErrors.department = "Select a department";
  if (!formData.salary) newErrors.salary = "Salary is required";
  if (!formData.confirm) newErrors.confirm = "Please confirm the form";
  setErrors(newErrors);
  return Object.keys(newErrors).length === 0;
};

// Handle submit
const handleSubmit = (e) => {

```

```

e.preventDefault();
if (!validate()) return;

if (editId !== null) {
  setEntries(
    entries.map((entry) => (entry.id === editId ? { ...formData, id: editId } : entry))
  );
  setEditId(null);
} else {
  setEntries([...entries, { ...formData, id: Date.now() }]);
}
setFormData(initialFormState);
};

// Handle edit
const handleEdit = (id) => {
  const entry = entries.find((e) => e.id === id);
  setFormData(entry);
  setEditId(id);
};

// Handle delete
const handleDelete = (id) => setEntries(entries.filter((e) => e.id !== id));

return (
  <div style={styles.container}>
    <h2>Employee Form</h2>
    <form onSubmit={handleSubmit} style={styles.form}>
      <label>
        Name:
        <input type="text" name="name" value={formData.name} onChange={handleChange} />
        {errors.name && <span style={styles.error}>{errors.name}</span>}
      </label>

```

<label>

Gender:

<select name="gender" value={formData.gender} onChange={handleChange}>

<option value="">Select Gender</option>

<option value="Male">Male</option>

<option value="Female">Female</option>

</select>

{errors.gender && {errors.gender}}

</label>

<label>

State:

<select name="state" value={formData.state} onChange={handleChange}>

<option value="">Select State</option>

{Object.keys(stateCityData).map((state) => (

<option key={state} value={state}>{state}</option>

)))

</select>

{errors.state && {errors.state}}

</label>

<label>

City:

<select name="city" value={formData.city} onChange={handleChange}>

<option value="">Select City</option>

{cities.map((city) => (

<option key={city} value={city}>{city}</option>

)))

</select>

{errors.city && {errors.city}}

</label>

<label>

Pincode:

```
<input type="text" name="pincode" value={formData.pincode} onChange={handleChange} />
{errors.pincode && <span style={styles.error}>{errors.pincode}</span>}
</label>
```

<label>

Email:

```
<input type="email" name="email" value={formData.email} onChange={handleChange} />
{errors.email && <span style={styles.error}>{errors.email}</span>}
</label>
```

<label>

Mobile:

```
<input type="text" name="mobile" value={formData.mobile} onChange={handleChange} />
{errors.mobile && <span style={styles.error}>{errors.mobile}</span>}
</label>
```

<label>

Department:

```
<select name="department" value={formData.department} onChange={handleChange}>
  <option value="">Select Department</option>
  {departments.map((dept) => (
    <option key={dept} value={dept}>{dept}</option>
  ))}
</select>
{errors.department && <span style={styles.error}>{errors.department}</span>}
</label>
```

<label>

Salary (USD):

```
<input type="number" name="salary" value={formData.salary} onChange={handleChange} />
{errors.salary && <span style={styles.error}>{errors.salary}</span>}
```

</label>

<label>

<input type="checkbox" name="confirm" checked={formData.confirm}
onChange={handleChange} />

Confirm all information

{errors.confirm && {errors.confirm}}

</label>

<button type="submit">{editId !== null ? "Update" : "Submit"}</button>

</form>

<h3>Employee List</h3>

{entries.length === 0 ? <p>No entries yet</p> : (

<table style={styles.table}>

<thead>

<tr>

<th>Name</th>

<th>Gender</th>

<th>State</th>

<th>City</th>

<th>Pincode</th>

<th>Email</th>

<th>Mobile</th>

<th>Department</th>

<th>Salary</th>

<th>Actions</th>

</tr>

</thead>

<tbody>

{entries.map((e) => (

<tr key={e.id}>

<td>{e.name}</td>

```

        <td>{e.gender}</td>
        <td>{e.state}</td>
        <td>{e.city}</td>
        <td>{e.pincode}</td>
        <td>{e.email}</td>
        <td>{e.mobile}</td>
        <td>{e.department}</td>
        <td>${e.salary}</td>
        <td>
            <button onClick={() => handleEdit(e.id)}>Edit</button>
            <button onClick={() => handleDelete(e.id)}>Delete</button>
        </td>
    </tr>
    )})
</tbody>
</table>
)}
</div>
);
}

```

```
// Simple styles
```

```

const styles = {
  container: { margin: "20px", fontFamily: "Arial, sans-serif" },
  form: { display: "flex", flexDirection: "column", gap: "10px", maxWidth: "400px" },
  error: { color: "red", fontSize: "12px", marginLeft: "5px" },
  table: { borderCollapse: "collapse", width: "100%", marginTop: "20px" },
  tableCell: { border: "1px solid #ccc", padding: "5px" },
};

```

```
export default CrudForm;
```

```

// *Build a toggle switch (on/off with state).
import React, { useState, useEffect } from "react";

function ToggleSwitch() {
  const [isOn, setIsOn] = useState(false);

  const handleToggle = () => setIsOn(prev => !prev);

  // Change body background based on toggle state
  useEffect(() => {
    document.body.style.backgroundColor = isOn ? "#d4f4dd" : "#f4d4d4"; // greenish for ON, reddish
    for OFF
  }, [isOn]);

  return (
    <div style={styles.container}>
      <span style={styles.label}>{isOn ? "ON" : "OFF"}</span>
      <div
        onClick={handleToggle}
        style={{
          ...styles.switch,
          backgroundColor: isOn ? "#4caf50" : "#ccc",
          justifyContent: isOn ? "flex-end" : "flex-start"
        }}
      >
        <div style={styles.knob}></div>
      </div>
    </div>
  );
}

```

```
// Inline styles
const styles = {
  container: {
    display: "flex",
    alignItems: "center",
    gap: "10px",
    fontFamily: "Arial, sans-serif",
    marginTop: "50px",
  },
  label: {
    fontSize: "18px",
    fontWeight: "bold",
  },
  switch: {
    width: "60px",
    height: "30px",
    borderRadius: "30px",
    display: "flex",
    alignItems: "center",
    padding: "3px",
    cursor: "pointer",
    transition: "background-color 0.3s, justify-content 0.3s",
  },
  knob: {
    width: "24px",
    height: "24px",
    borderRadius: "50%",
    backgroundColor: "#fff",
    transition: "0.3s",
  },
};
```

```
export default ToggleSwitch;
```

```
// *Create a searchable list that filters items based on user input.
```

```
import React, { useState, useEffect } from "react";
```

```
function SearchableApiList() {
```

```
  const [items, setItems] = useState([]);
```

```
  const [searchTerm, setSearchTerm] = useState("");
```

```
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] = useState(null);
```

```
  // Fetch data from API
```

```
  useEffect(() => {
```

```
    const fetchData = async () => {
```

```
      try {
```

```
        const response = await fetch("https://jsonplaceholder.typicode.com/users");
```

```
        if (!response.ok) throw new Error("Failed to fetch data");
```

```
        const data = await response.json();
```

```
        setItems(data);
```

```
      } catch (err) {
```

```
        setError(err.message);
```

```
      } finally {
```

```
        setLoading(false);
```

```
      }
```

```
    };
```

```
    fetchData();
```

```
  }, []);
```

```
  // Filter items by name based on search term
```

```
  const filteredItems = items.filter((item) =>
```

```
    item.name.toLowerCase().includes(searchTerm.toLowerCase())
```

```
  );
```

```
if (loading) return <p>Loading...</p>;  
if (error) return <p>Error: {error}</p>;
```

```
return (  
  <div style={styles.container}>  
    <h2>Search Users</h2>  
    <input  
      type="text"  
      placeholder="Search by name..."  
      value={searchTerm}  
      onChange={(e) => setSearchTerm(e.target.value)}  
      style={styles.input}  
    />
```

```
    <ul style={styles.list}>  
      {filteredItems.length > 0 ? (  
        filteredItems.map((item) => (  
          <li key={item.id}>  
            <strong>{item.name}</strong> - {item.email}  
          </li>  
        ))  
      ) : (  
        <li>No users found</li>  
      )}  
    </ul>  
  </div>
```

```
);  
}
```

```
// Inline styles
```

```
const styles = {  
  container: { margin: "20px", fontFamily: "Arial, sans-serif" },
```

```
input: { padding: "8px", width: "250px", marginBottom: "10px" },  
list: { listStyleType: "none", paddingLeft: 0 },  
};
```

```
export default SearchableApiList;
```

```
// *Implement a tabbed UI using conditional rendering.
```

```
import React, { useState } from "react";
```

```
function TabbedUI() {  
  const [activeTab, setActiveTab] = useState("home");  
  
  return (  
    <div style={styles.container}>  
      <h2>Tabbed UI Example</h2>  
  
      { /* Tab Buttons */}  
      <div style={styles.tabContainer}>  
        <button  
          style={activeTab === "home" ? styles.activeTab : styles.tab}  
          onClick={() => setActiveTab("home")}  
        >  
          Home  
        </button>  
        <button  
          style={activeTab === "profile" ? styles.activeTab : styles.tab}  
          onClick={() => setActiveTab("profile")}  
        >  
          Profile  
        </button>  
        <button  
          style={activeTab === "settings" ? styles.activeTab : styles.tab}  
          onClick={() => setActiveTab("settings")}  
        >  
          Settings  
        </button>  
      </div>  
    </div>  
  );  
}
```



```

>
  Settings
</button>
</div>

{/* Tab Content */}
<div style={styles.content}>
  {activeTab === "home" && <p>Welcome to the Home tab!</p>}
  {activeTab === "profile" && <p>This is your Profile information.</p>}
  {activeTab === "settings" && <p>Adjust your Settings here.</p>}
</div>
</div>
);
}

```

// Inline styles

```

const styles = {
  container: { margin: "20px", fontFamily: "Arial, sans-serif" },
  tabContainer: { display: "flex", gap: "10px", marginBottom: "20px" },
  tab: {
    padding: "10px 20px",
    cursor: "pointer",
    backgroundColor: "#eee",
    border: "none",
    borderRadius: "5px",
  },
  activeTab: {
    padding: "10px 20px",
    cursor: "pointer",
    backgroundColor: "#4caf50",
    color: "#fff",
    border: "none",
    borderRadius: "5px",
  },

```

```
},  
content: { padding: "20px", border: "1px solid #ccc", borderRadius: "5px" },  
};
```

```
export default TabbedUI;
```

```
// *Build a form with validation (e.g., email, password).
```

```
import React, { useState } from "react";
```

```
function LoginForm() {  
  const [formData, setFormData] = useState({ email: "", password: "" });  
  const [errors, setErrors] = useState({});  
  const [showPassword, setShowPassword] = useState(false); // toggle state  
  
  const handleChange = (e) => {  
    const { name, value } = e.target;  
    setFormData({ ...formData, [name]: value });  
  };  
  
  const validate = () => {  
    const newErrors = {};  
    if (!formData.email) {  
      newErrors.email = "Email is required";  
    } else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(formData.email)) {  
      newErrors.email = "Invalid email address";  
    }  
  
    if (!formData.password) {  
      newErrors.password = "Password is required";  
    } else if (formData.password.length < 6) {  
      newErrors.password = "Password must be at least 6 characters";  
    }  
  }  
}
```

```
setErrors(newErrors);

return Object.keys(newErrors).length === 0;

};
```

```
const handleSubmit = (e) => {

  e.preventDefault();

  if (validate()) {

    alert(`Login successful!\nEmail: ${formData.email}`);

    setFormData({ email: "", password: "" });

  }

};
```

```
return (

  <div style={styles.container}>

    <h2>Login Form</h2>

    <form onSubmit={handleSubmit} style={styles.form}>

      <label>

        Email:

        <input

          type="email"

          name="email"

          value={formData.email}

          onChange={handleChange}

          style={styles.input}

        />

        {errors.email && <span style={styles.error}>{errors.email}</span>}

      </label>

      <label style={{ position: "relative" }}>

        Password:

        <input

          type={showPassword ? "text" : "password"}

          name="password"
```

```

    value={formData.password}

    onChange={handleChange}

    style={{ ...styles.input, paddingRight: "30px" }}
  />
  <span
    onClick={() => setShowPassword(prev => !prev)}
    style={styles.eyeIcon}
  >
    {showPassword ? "👁️" : "🔒"}
  </span>
  {errors.password && <span style={styles.error}>{errors.password}</span>}
</label>

  <button type="submit" style={styles.button}>Login</button>
</form>
</div>
);
}

// Inline styles
const styles = {
  container: { margin: "20px", fontFamily: "Arial, sans-serif" },
  form: { display: "flex", flexDirection: "column", gap: "15px", maxWidth: "300px" },
  input: { padding: "8px", marginTop: "5px", width: "100%" },
  button: { padding: "10px", backgroundColor: "#4caf50", color: "#fff", border: "none", cursor: "pointer" },
  error: { color: "red", fontSize: "12px" },
  eyeIcon: {
    position: "absolute",
    right: "10px",
    top: "35px",
    cursor: "pointer",
    fontSize: "18px",
  }
};

```

```

    userSelect: "none",
  },
};

export default LoginForm;

// *Build a star rating component with hover and click.
import React, { useState } from "react";

function StarRating({ totalStars = 5 }) {
  const [rating, setRating] = useState(0);    // selected rating
  const [hover, setHover] = useState(0);     // hover rating

  return (
    <div style={styles.container}>
      <h2>Star Rating</h2>
      <div style={styles.stars}>
        {Array.from({ length: totalStars }, (_, index) => {
          const starValue = index + 1;
          return (
            <span
              key={index}
              style={{
                ...styles.star,
                color: starValue <= (hover || rating) ? "#ffc107" : "#ccc",
              }}
              onClick={() => setRating(starValue)}
              onMouseEnter={() => setHover(starValue)}
              onMouseLeave={() => setHover(0)}
            >
              ★
            </span>
          );
        })}
      </div>
    </div>
  );
}

```

```

    }}
  </div>
  <p>Your Rating: {rating} / {totalStars}</p>
</div>

);
}

```

// Inline styles

```

const styles = {
  container: { margin: "20px", fontFamily: "Arial, sans-serif" },
  stars: { display: "flex", gap: "5px", fontSize: "40px", cursor: "pointer" },
  star: { transition: "color 0.2s" },
};

```

export default StarRating;

// *Build a custom dropdown using keyboard navigation.

// *Create a reusable modal component with backdrop and escape handling.

model.jsx

```
import React, { useState } from "react";
```

```
import Modal from "../Modal";
```

```
function App() {
```

```
  const [isModalOpen, setIsModalOpen] = useState(false);
```

```
  return (
```

```
    <div style={{ padding: "20px" }}>
```

```
      <h2>Reusable Modal Example</h2>
```

```
      <button
```

```
        onClick={() => setIsModalOpen(true)}
```

```
        style={{ padding: "10px 20px", cursor: "pointer" }}
```

```
      >
```

```
        Open Modal
```

```
</button>
```

```
<Modal isOpen={isOpen} onClose={() => setIsModalOpen(false)}>
```

```
<h3>Modal Content</h3>
```

```
<p>This is a reusable modal component with backdrop and escape handling.</p>
```

```
</Modal>
```

```
</div>
```

```
);
```

```
}
```

```
export default App;
```

App.jsx

```
import React, { useEffect } from "react";
```

```
function Modal({ isOpen, onClose, children }) {
```

```
  // Close on Escape key
```

```
  useEffect(() => {
```

```
    const handleKeyDown = (e) => {
```

```
      if (e.key === "Escape") {
```

```
        onClose();
```

```
      }
```

```
    };
```

```
    if (isOpen) {
```

```
      document.addEventListener("keydown", handleKeyDown);
```

```
    }
```

```
    return () => document.removeEventListener("keydown", handleKeyDown);
```

```
  }, [isOpen, onClose]);
```

```
  if (!isOpen) return null;
```

```
  return (
```

```
    <div style={styles.backdrop} onClick={onClose}>
```

```

<div
  style={styles.modal}
  onClick={(e) => e.stopPropagation()} // prevent closing when clicking inside modal
>
  {children}
  <button onClick={onClose} style={styles.closeButton}>
    Close
  </button>
</div>
</div>
);
}

```

// Styles

```

const styles = {
  backdrop: {
    position: "fixed",
    top: 0,
    left: 0,
    right: 0,
    bottom: 0,
    backgroundColor: "rgba(0,0,0,0.5)",
    display: "flex",
    justifyContent: "center",
    alignItems: "center",
    zIndex: 1000,
  },
  modal: {
    backgroundColor: "#fff",
    padding: "20px",
    borderRadius: "8px",
    minWidth: "300px",
    maxWidth: "500px",
  }
}

```



```
    boxShadow: "0 2px 10px rgba(0,0,0,0.3)",
    position: "relative",
  },
  closeButton: {
    marginTop: "20px",
    padding: "8px 16px",
    backgroundColor: "#4caf50",
    color: "#fff",
    border: "none",
    cursor: "pointer",
    borderRadius: "4px",
  },
};
```

```
export default Modal;
```

```
// *Implement infinite scroll or pagination using an API.
```

```
import React, { useState, useEffect, useRef, useCallback } from "react";
```

```
function InfiniteScrollList() {
  const [posts, setPosts] = useState([]);
  const [page, setPage] = useState(1);
  const [hasMore, setHasMore] = useState(true);
  const [loading, setLoading] = useState(false);
```

```
  const observer = useRef();
```

```
  // Fetch posts from API
```

```
  const fetchPosts = async (page) => {
    setLoading(true);
    try {
```

```

const res = await fetch(`https://jsonplaceholder.typicode.com/posts?_limit=10&_page=${page}`);
const data = await res.json();
if (data.length === 0) setHasMore(false);
setPosts((prev) => [...prev, ...data]);
} catch (err) {
  console.error(err);
} finally {
  setLoading(false);
}
};

```

```

useEffect(() => {
  fetchPosts(page);
}, [page]);

```

// Observer for last element

```

const lastPostRef = useCallback(
  (node) => {
    if (loading) return;
    if (observer.current) observer.current.disconnect();

    observer.current = new IntersectionObserver((entries) => {
      if (entries[0].isIntersecting && hasMore) {
        setPage((prev) => prev + 1);
      }
    });
  },

```

```

  if (node) observer.current.observe(node);
},
[loading, hasMore]
);

```

```

return (

```

```

<div style={styles.container}>
  <h2>Infinite Scroll Posts</h2>
  <ul style={styles.list}>
    {posts.map((post, index) => {
      if (posts.length === index + 1) {
        return (
          <li ref={lastPostRef} key={post.id} style={styles.item}>
            <strong>{post.id}. {post.title}</strong>
          </li>
        );
      } else {
        return (
          <li key={post.id} style={styles.item}>
            <strong>{post.id}. {post.title}</strong>
          </li>
        );
      }
    })}
  </ul>
  {loading && <p>Loading...</p>}
  {!hasMore && <p>No more posts</p>}
</div>
);
}

```

// Inline styles

```

const styles = {
  container: { margin: "20px", fontFamily: "Arial, sans-serif" },
  list: { listStyle: "none", padding: 0 },
  item: { padding: "10px", borderBottom: "1px solid #ccc" },
};

```

```

export default InfiniteScrollList;

```

// *Drag-and-drop functionality with custom hooks.